

// proto • mcp • kubernetes • deckhouse

как дать агенту управлять кластером вашей инфры?

Эдгар Сипки // Founder EasyP && SIPKI Tech

// speaker.bio

кто я и где мы

01

эдгар сипки

Senior golang developer, open-source. Founder **EasyP**.

02

наш контекст

Маленький стартап. Мало денег, нет инфраструктурной команды.

// agenda

о чём поговорим?

- 01 Агент и MCP за пять минут
- 02 Стартап против инфраструктуры
- 03 Deckhouse и первый MCP-сервер
- 04 spec-first и protoc-gen-mcp
- 05 Сценарии и живое демо

// chapter 01 / starting

о чём поговорим?

- 01 **Агент и MCP за пять минут**
- 02 Стартап против инфраструктуры
- 03 Deckhouse и первый MCP-сервер
- 04 spec-first и protoc-gen-mcp
- 05 Сценарии и живое демо

// chapter 01

агент и мср

// agent.intro

LLM умна, но безрука

- Модель отлично рассуждает
- Но сама по себе ничего не делает в реальном мире
- → чтобы действовать, ей нужны **руки**

// agent.anatomy

анатомия агента

LLM

рассуждает и принимает решения

ЦИКЛ

план → действие → наблюдение

инструменты

руки во внешний мир

контекст

что модель видит сейчас

// mcp.what

mcp = USB-C для интеграций

- Один протокол вместо зоопарка кастомных коннекторов
- Агент видит и вызывает внешние действия через один стандарт
- **MCP-сервер = руки агента**

// mcp.primitives

три примитива

TOOLS 

действия

resources 

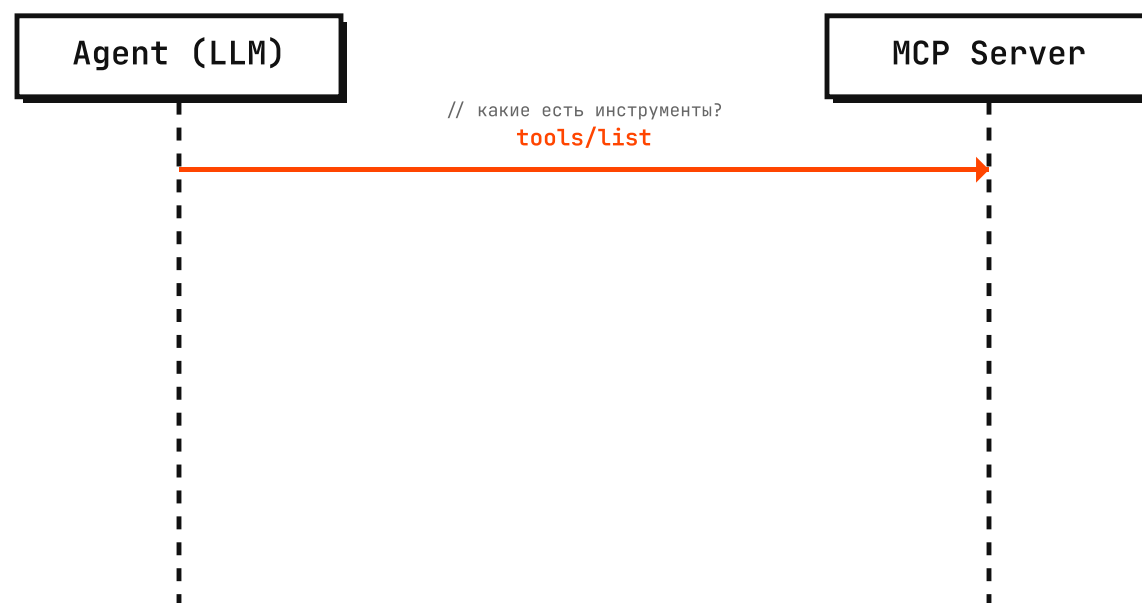
чтение

prompts 

шаблоны

// mcp.flow

как агент вызывает tool



→ description и JSON Schema решают, сможет ли агент вообще вызвать tool

// chapter 02 / starting

о чём поговорим?

→ ~~01 Агент и MCP за пять минут~~

→ 02 **Стартап против инфраструктуры**

→ 03 Deckhouse и первый MCP-сервер

→ 04 spec-first и protoc-gen-mcp

→ 05 Сценарии и живое демо

// chapter 02

стартан против инфраструктуры

// task

наша задача

- Нужно управлять Kubernetes-кластером
- Инфраструктурной экспертизы **нет**
- Денег на DevOps-команду — тоже нет

// economics

вилка ресурсов

// конкурент

\$93M

ИНВЕСТИЦИЙ

// мы

500к ₺

И ТО МОИ ЛИЧНЫЕ

// infra: h3llo.cloud – облако без квотов

// the.bet

единственная разумная ставка

Не нанимать DevOps, а **переложить управление кластером на агента**.

// attempt.01

попытка №1: голый k8s-mcp



// тулы:



→ выглядит просто... но только на первый взгляд

// why.fail

почему не взлетело

- Чтобы управлять k8s-MCP, надо самому знать k8s
- CRD , RBAC , манифесты, неймспейсы
- → инструмент **переложил сложность на нас**, а не убрал её **✗**

// chapter 03 / starting

о чём поговорим?

- ~~01 Агент и MCP за пять минут~~
- ~~02 Старт против инфраструктуры~~
- **03 Deckhouse и первый MCP-сервер**
- 04 spec-first и protoc-gen-mcp
- 05 Сценарии и живое демо

// chapter 03

deckhouse и первый mcp-сервер

// deckhouse.what

услышали про deckhouse

- Готовая Kubernetes-платформа
- Подняли кластер на **h3llo.cloud** — без квотов с биллингом
- Берёт много инфраструктурной рутины на себя
- «А почему бы и нет?» — взяли, потестили

// deckhouse.verdict

вердикт



мы

инструмент хороший, но мы для него ещё слишком глупы



агент

а агент — умный

// deckhouse.idea

Новая идея

Deckhouse держит платформу, агент **разговаривает с ней за нас**.

→ нужен мост: MCP-сервер для Deckhouse

// build.start

пишем mcp-сервер с агентом

- Сели писать MCP-сервер для Deckhouse
- Агент помогает кодить
- Иии?

// build.problem

барьер не там, где ждали

Не инфраструктура, а инженерия: агенту **не хватало структуры.**

// build.symptom

сумнмом

// без жёсткого контракта

агент лезет переписывать уже готовые куски

регрессии · дрейф · «я тут немного поправил»

// build.diagnosis

диагноз

- Нужен источник истины, который агент не имеет права трогать
- Код для этого — **плохое место**

// chapter 04 / starting

о чём поговорим?

- ~~01 Агент и MCP за пять минут~~
- ~~02 Стартап против инфраструктуры~~
- ~~03 Deckhouse и первый MCP сервер~~
- 04 **spec-first и protoc-gen-mcp**
- 05 Сценарии и живое демо

// chapter 04

spec-first u protoc-gen-mcp

// spec.search

угём за spec-first

Хотим: спека как **контракт, который агент не может сломать.**

// spec.candidates

нашли два варианта

// что хотели

- Полная поддержка примитивов MCP
- Аннотации и валидация
- Типизация

// что получили

- Сильно ограничены в поддержке спецификации
- Нет части примитивов
- Не покрывают наш кейс

// spec.solution

делаем свой

- Мы проектируем спецификацию (proto-контракт аннотаций)
- Генератор пишет агент

// the.meta-thesis

агент ужасно проектирует,
но неплохо кодит

человек держит спеку и архитектуру · агент пишет реализацию

// protoc-gen-mcp.idea

protoc-gen-mcp

- protobuf — единственный источник истины
- Аннотации в `.proto` описывают tools / prompts / resources
- Генератор → **типизированные MCP-биндинги + JSON Schema**

// options.proto

аннотации в .proto

```
1  service DiagnosticsAPI {
2      option (mcp.options.v1.service) = { namespace: "deckhouse" };
3
4      // Агрегированный статус кластера: ноды, модули, релизы.
5      rpc GetClusterStatus(google.protobuf.Empty)
6          returns (GetClusterStatusResponse) {
7          option (mcp.options.v1.method) = {
8              annotations: { read_only_hint: true }
9          };
10     }
11 }
```

→ read_only_hint, валидация и description — всё в контракте

// why.it.works

почему это лечит «структуру»

→ Контракт живёт в `.proto`, а не в редактируемом коде

→ → случайной правкой структуру не разнести

```
// deckhose-mcp / handler
```

сгенерированный интерфейс

```
1 // Сгенерированный интерфейс — агент его не трогает.  
2 type DiagnosticsAPIToolHandler interface {  
3     GetClusterStatus(ctx context.Context,  
4         req *emptypb.Empty) (*GetClusterStatusResponse, error)  
5     ListNodes(ctx context.Context,  
6         req *ListNodesRequest) (*ListNodesResponse, error)  
7     // ...  
8 }
```

// easyp.yaml

генерация через easyp

```
1 generate:
2   inputs:
3     - directory: { path: proto, root: "." }
4   plugins:
5     - name: go
6       out: .
7       opts: { paths: source_relative }
8     - command: ["go", "run",
9       "github.com/easyp-tech/protoc-gen-mcp/cmd/protoc-gen-mcp@latest"]
10      out: .
11      opts: { paths: source_relative }
```

→ `task generate` = easyp mod download + easyp generate

// main.go

бизнес-логика — в хендлере

```
1 server := mcp.NewServer("deckhouse-mcp", "v1")
2 deckhousev1.RegisterDiagnosticsAPITools(server, handler)
3 server.ServeSSE(":8080")
```

// chapter 05 / starting

о чём поговорим?

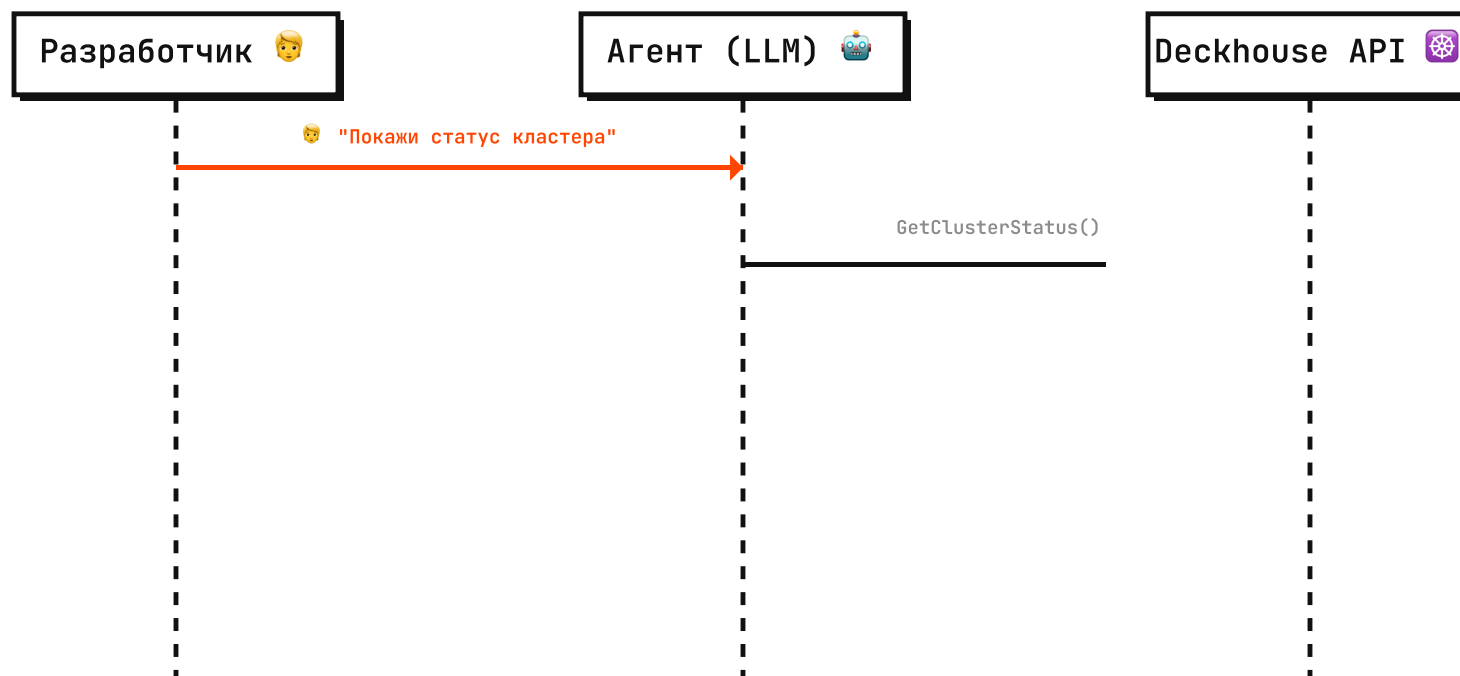
- ~~01 Агент и MCP за пять минут~~
- ~~02 Стартап против инфраструктуры~~
- ~~03 Deckhouse и первый MCP-сервер~~
- ~~04 spec first и protoc-gen-mcp~~
- **05 Сценарии и живое демо**

// chapter 05

сценарии и гемо

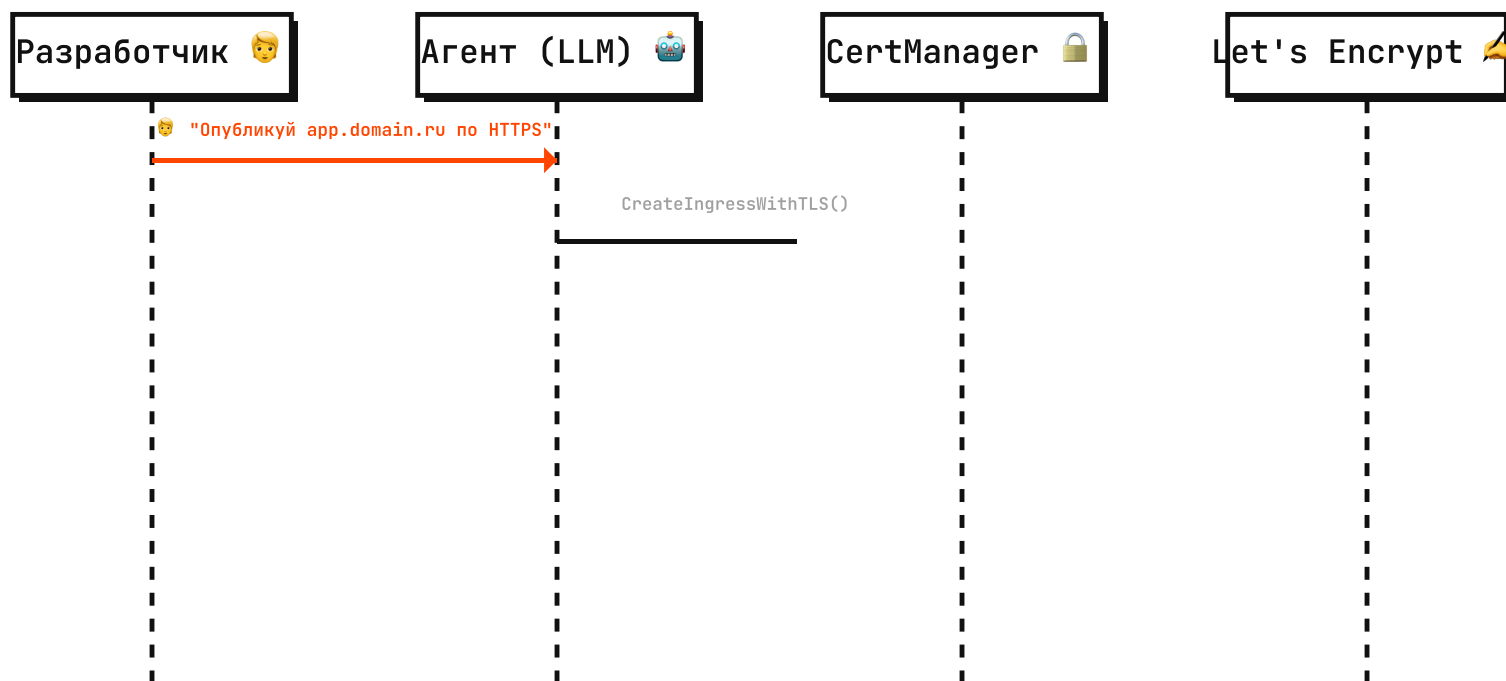
// scenario.a / diagnostics

сценарий а — диагностика



// scenario.c / ingress-tls

сценарий с — https ingress + TLS



// live demo

агент управляет кластером



opencode

Ask anything... "Fix broken tests"

Build · Big Pickle OpenCode Zen

tab agents ctrl+p commands

• Tip Run /compact to summarize long sessions near context limits

▶ 0:00 / 4:57



// demo.discussion

что важно заметить

// где сработала спека

- Агент сделал ровно то, что описано
- Нет импровизации в схемах

// где помог контракт

- Структура в .proto не даёт разнести код
- Регенерация вместо ручных правок

// summary

5 тезисов

Агенту нужны руки

MCP — это руки

Готовая платформа (Deckhouse) бьёт самописную интеграцию

Агент без структуры разносит код

spec-first (protoc-gen-mcp) = **контракт, который агент не ломает**

// when

когда этот путь оправдан

// да

- Маленькая команда без инфры + правильное облако (h3llo.cloud)
- Нужно много, ресурса мало

// спорно

- Зрелый DevOps с экспертизой
- Критичные destructive-операции без человека

// roadmap

что дальше?

// roadmap

куда движемся

now

Tools / Prompts / Resources · Go, Python, Kotlin, Java, TypeScript

next

Больше валидации и ToolAnnotations по MCP spec 2025-11-25

later

Resources для всех языков · Agent Skill для авторинга

// спасибо • вопросы • github

protoc-gen-mcp



protoc-gen-mcp



deckhouse-mcp

Эдгар Сипки // Founder EasyP

// community.easyp

easyp community



<https://t.me/+V0c8f4PsnJEzMDQy>